



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Food Rescue Connect: Smart Food Donation and Redistribution Management System

Industry Context

A CASE STUDY REPORT

Submitted in the partial fulfilment for the Course of

CSA1016-Software Engineering

to the award of the degree of

BACHELOR OF ENGINEERING

IN

CSE-AI

Submitted by

Sk. Ashwaq Ahamed (192572070)

Under the Supervision of

Dr. K. ANITA DAVAMANI

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

ABSTRACT

FoodRescue Connect is a proposed digital platform that connects food donors, recipient organizations, volunteers, and administrators to reduce food wastage and improve the redistribution of surplus food. Since the application contains multiple user roles and time-sensitive workflows, a reliable software delivery process is required to ensure that every code change is built, tested, reviewed, secured, and deployed consistently.

This document designs a Continuous Integration and Continuous Deployment (CI/CD) pipeline for FoodRescue Connect. The pipeline automates source-code validation, dependency installation, compilation or application build, unit testing, integration testing, code-quality analysis, security scanning, packaging, deployment to development and staging environments, acceptance testing, production release, monitoring, and rollback.

A Git-based source-control workflow is proposed with GitHub as the repository platform and GitHub Actions as the automation engine. Docker is proposed for reproducible packaging, while a container registry stores approved images. Automated tests are divided into unit, API/integration, end-to-end, security, and smoke tests. A staging environment is used as a production-like validation layer before release.

The design also includes branch protection, secret management, dependency scanning, container scanning, deployment approvals, notifications, health checks, and rollback mechanisms. The proposed architecture aims to make delivery repeatable, traceable, secure, and reliable while allowing the FoodRescue Connect development team to release improvements quickly.

1. INTRODUCTION

CI/CD is a software engineering practice that automates the movement of source code through integration, verification, packaging, and deployment stages. Continuous Integration encourages developers to integrate small changes frequently so that defects are detected early. Continuous Delivery or Deployment extends this process by automatically preparing and releasing validated software to appropriate environments.

For FoodRescue Connect, manual deployment would create avoidable risks. The application may receive changes to registration, donation management, search, requests, volunteer assignment, notifications, or administration. If these changes are deployed manually, a missing dependency, incorrect configuration, failed migration, or untested feature could affect users.

The CI/CD pipeline therefore acts as a quality gate between source code and production. A change is allowed to progress only when required checks pass. Failed tests stop the pipeline, while successful builds produce a versioned deployable artifact. This makes it possible to identify which source commit produced a release and to return to a previous known-good version when necessary.

The proposed design is technology-neutral at the application level but uses a practical toolchain for implementation: GitHub for source control and pull requests, GitHub Actions for pipeline automation, a package manager for dependency management, Docker for containerization, static analysis and security scanning tools for quality gates, and separate development, staging, and production environments.

2. CI/CD REQUIREMENTS ANALYSIS

2.1 Project Characteristics

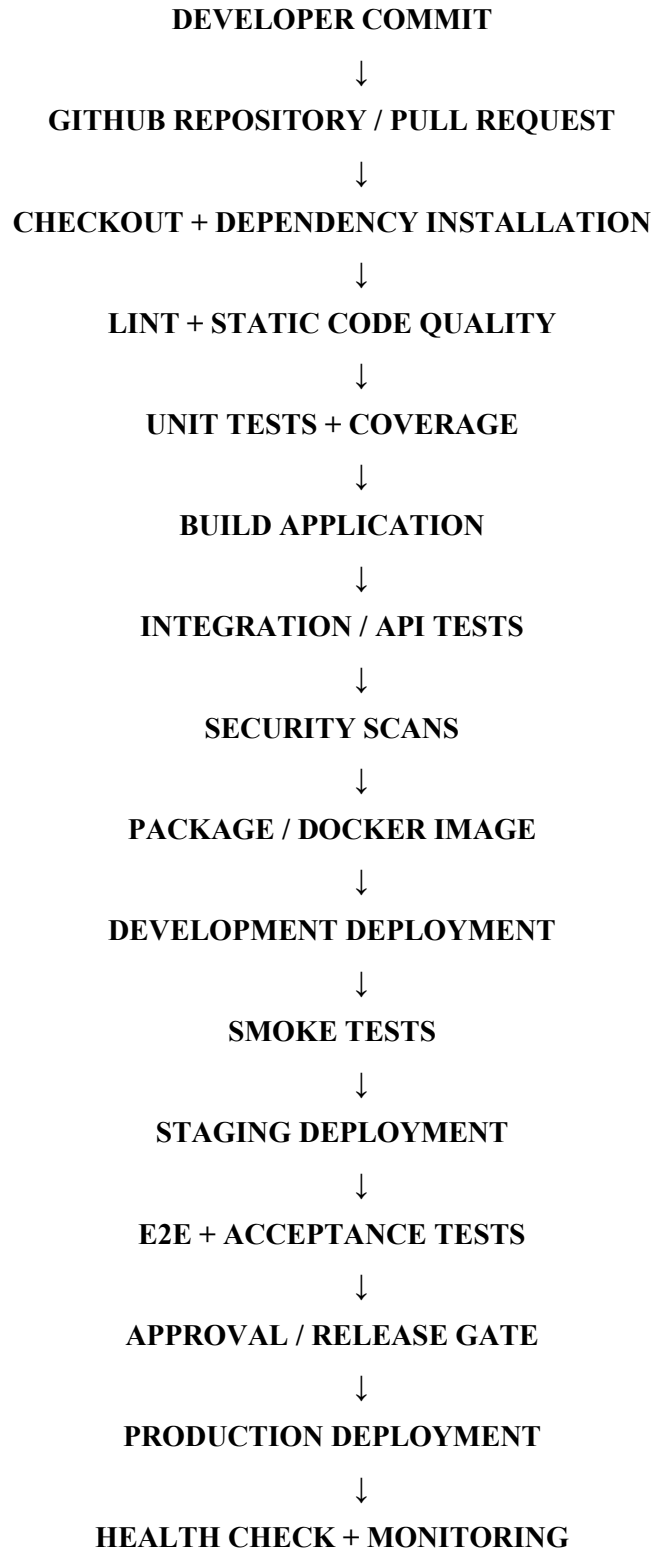
- Multiple user roles: donor, recipient, volunteer, and administrator.
- Core workflows include registration, authentication, food donation, search, requests, approval, pickup, delivery, notifications, and administration.
- The application may contain a web frontend, backend/API services, and a persistent database.
- Changes must be validated before deployment because incorrect logic can affect food availability, requests, and delivery status.
- The deployment process should be repeatable and should minimize service interruption.
- Sensitive information such as database credentials, API keys, and deployment secrets must not be stored directly in source code.

2.2 CI/CD Needs

ID	Need	Requirement
N01	Fast feedback	Run linting and unit tests on every pull request.
N02	Reliable integration	Build and test the application automatically after changes are merged.
N03	Consistent packaging	Create a versioned container/image or deployable artifact.
N04	Quality control	Perform static analysis and maintain agreed quality thresholds.
N05	Security	Scan dependencies, source code, and container images.
N06	Environment separation	Use development, staging, and production environments.
N07	Safe production release	Require successful staging tests and approval before production.
N08	Recovery	Keep previous versions available for rapid rollback.
N09	Observability	Send deployment notifications and perform post-deployment health checks.

3. CI/CD PIPELINE ARCHITECTURE

The proposed pipeline follows a left-to-right flow from developer commit to monitored production release. Each stage acts as a gate. A failure in a mandatory stage stops promotion so that defective code does not continue to later environments.





ROLLBACK IF REQUIRED

Figure 1: Proposed CI/CD pipeline workflow for FoodRescue Connect.

3.1 Pipeline Stages

Stage	Purpose	Suggested Tool	Main Activity
1	Source Control	GitHub	Commit, branch, pull request, review
2	Build Preparation	GitHub Actions + package manager	Install dependencies and prepare build
3	Code Quality	ESLint/Flake8/SonarQube or equivalent	Linting and static analysis
4	Unit Testing	Jest/PyTest/JUnit or equivalent	Verify individual components
5	Build	Docker/build tool	Create repeatable application build
6	Integration Testing	API/database test framework	Verify service interactions
7	Security	Dependabot/Snyk/Trivy or equivalent	Dependency and image vulnerability checks
8	Packaging	Docker + container registry	Create versioned image/artifact
9	Deployment	Container platform/cloud environment	Deploy to dev, staging, production
10	Verification	Smoke/E2E/health checks	Confirm deployment works
11	Monitoring	Logs, metrics, alerts	Detect post-release problems

4. TOOLS AND TECHNOLOGIES

Tool/Technology	Stage	Justification
GitHub	Source control and collaboration	Branches, pull requests, code review, history, branch protection.
GitHub Actions	CI/CD automation	Native workflow automation triggered by commits and pull requests.
Docker	Packaging	Creates consistent container images across environments.
Container Registry	Artifact storage	Stores versioned images that have passed pipeline checks.
Jest / PyTest / JUnit	Automated testing	Runs unit tests depending on the chosen application stack.
Postman/Newman or API framework	API testing	Automates backend endpoint and integration validation.
Playwright/Cypress	End-to-end testing	Validates important user journeys in a browser.
SonarQube / equivalent	Code quality	Static analysis, maintainability checks, and quality gates.
Trivy / Snyk / Dependabot	Security	Scans dependencies and container images for vulnerabilities.
Docker Compose / test database	Integration environment	Provides repeatable service and database dependencies.
Cloud/container platform	Deployment	Hosts development, staging, and production services.
Slack/Email notifications	Notifications	Alerts team about failures, releases, and rollback events.

The exact tools may be adjusted to the programming language and hosting platform selected for the FoodRescue Connect implementation. The important principle is that each stage should be automated, reproducible, observable, and connected to a clear quality gate.

5. AUTOMATED TESTING STRATEGY

Automated testing is a central part of the pipeline because the system contains multiple workflows and user roles. Testing is organized into layers so that fast tests run early and broader tests run after a valid build is available.

Test Layer	When	Examples	Gate
Unit Tests	Every pull request	Validate validation logic, authentication helpers, donation calculations, status rules, and service functions.	Must pass before merge.
API/Integration Tests	After build	Verify frontend/backend contracts, database operations, request processing, and notifications.	Mandatory quality gate.
End-to-End Tests	Staging	Verify critical user journeys such as registration, donation creation, search, request, approval, and delivery.	Required before production.
Security Tests	Every build/release	Dependency, secret, static and container vulnerability checks.	Block release for critical findings according to policy.
Smoke Tests	After each deployment	Check service availability, login, health endpoint, and one core workflow.	Immediate post-deployment gate.
Regression Tests	Release/staging	Verify previously working functionality after major changes.	Required for release candidates.

5.1 Suggested Automated Scenarios

- A donor can register, log in, and create a valid food donation.
- Invalid donation quantities and expired food records are rejected.
- A recipient can search available donations and submit a valid request.
- A request cannot be approved when the donation is unavailable or expired.
- Volunteer assignment cannot create conflicting assignments.
- Status transitions follow the defined workflow.
- Unauthorized users cannot access administrative endpoints.
- Critical APIs return successful responses and correct error codes.
- Production health checks pass after deployment.

6. DEPLOYMENT STRATEGY

Three primary environments are proposed: Development, Staging, and Production. Separating environments prevents incomplete changes from directly reaching end users and provides a controlled area for final verification.

Environment	Promotion	Purpose	Data/Configuration
Development	Automatic after successful CI on the main development branch	Developer testing, integration checks	Non-production database and test data.
Staging	Automatic after development verification or release candidate creation	Production-like acceptance and E2E testing	Configuration closely matching production.
Production	Manual approval after staging gates pass	Real users and operational workload	Production database and protected secrets.

6.1 Deployment Flow

- Build a versioned artifact from the approved source commit.
- Deploy the artifact to Development.
- Run smoke and integration checks.
- Promote the same artifact to Staging rather than rebuilding it.
- Run end-to-end, acceptance, and security checks.
- Require release approval from an authorized maintainer.
- Deploy the exact tested artifact to Production.
- Run production health checks and monitor logs/metrics.
- If critical checks fail, trigger rollback to the last known-good release.

6.2 Deployment Strategy Choice

A rolling or blue-green style deployment can be used depending on the hosting platform. For an academic implementation, a rolling deployment is simpler. For higher availability, blue-green deployment is preferable because the new version can be validated before traffic is switched. The pipeline should always preserve the previous production artifact for rollback.

7. SECURITY, QUALITY, NOTIFICATIONS AND ROLLBACK

7.1 Security Controls

- Store passwords and credentials using secure secrets management rather than source files.
- Use repository branch protection and require pull-request review before merging to protected branches.
- Scan third-party dependencies for known vulnerabilities.
- Scan Docker/container images before promotion.
- Use least-privilege permissions for CI/CD service accounts.
- Prevent secrets from being printed in logs.
- Use HTTPS/TLS for deployed services and secure environment configuration.
- Run authorization tests for donor, recipient, volunteer, and administrator roles.

7.2 Quality Gates

Gate	Condition	Action
Build	Application builds successfully	Stop pipeline if build fails
Unit tests	All mandatory unit tests pass	Stop pipeline
Coverage	Meets project-defined threshold	Warn or stop based on policy
Static analysis	No blocking critical quality issues	Stop pipeline
Security	No unaccepted critical vulnerabilities	Stop pipeline
Staging	Smoke/E2E tests pass	Stop promotion
Production health	Health endpoint and critical workflow pass	Rollback if failed

7.3 Notifications

The pipeline should notify the development team when a build fails, a pull request passes required checks, a staging deployment completes, a production release succeeds, or a rollback is performed. Notifications should contain the project, branch or release identifier, stage, status, and a link to the pipeline log.

7.4 Rollback Mechanism

Every production release should be versioned. If a health check or monitoring alert indicates a serious failure, the deployment controller can restore the previous known-good version. Database changes should use backward-compatible migration practices so that application rollback does not leave the database in an unusable state.

8. PIPELINE CONFIGURATION AND WORKFLOW

A GitHub Actions workflow can be organized into jobs such as quality, test, build, security, package, deploy-development, deploy-staging, and deploy-production. Jobs should depend on earlier jobs so that a failure prevents unsafe promotion.

8.1 Example Workflow Logic

Step	Job/Stage	Action
Trigger	Push or pull request	Start validation for source changes.
Checkout	Retrieve repository	Use the exact commit associated with the workflow.
Setup	Install runtime and dependencies	Use a locked dependency file where applicable.
Quality	Lint + static analysis	Reject formatting and critical quality problems.
Unit Test	Run automated tests	Publish test results and coverage.
Build	Compile/package application	Fail on build errors.
Integration	Start test services and execute API tests	Validate service/database interactions.
Security	Dependency + image scans	Block critical unresolved vulnerabilities.
Package	Build and tag artifact/image	Tag with commit SHA or release version.
Deploy Dev	Deploy artifact	Run smoke tests.
Deploy Staging	Promote same artifact	Run E2E and acceptance tests.
Approve	Authorized approval	Open production gate.
Deploy Prod	Deploy tested artifact	Run health checks.
Observe	Monitor	Rollback if defined failure conditions occur.

8.2 Branching Strategy

- feature/* branches are used for individual changes.
- Pull requests merge reviewed changes into the development/integration branch.
- The main/release branch is protected and contains production-ready code.
- Release tags identify production versions and support traceability and rollback.

9. REQUIREMENT TRACEABILITY AND EXPECTED BENEFITS

Requirement	CI/CD Area	Implementation	Expected Benefit
R1	Source control	GitHub + protected branches	Every change is traceable.
R2	Automated build	GitHub Actions + Docker	Repeatable build process.
R3	Automated testing	Unit/API/E2E tests	Defects detected before release.
R4	Quality analysis	Static analysis	Improved maintainability.
R5	Security	Dependency/container scans	Reduced security risk.
R6	Environment separation	Dev/Staging/Production	Safer promotion.
R7	Deployment	Automated deployment jobs	Reduced manual errors.
R8	Release approval	Protected production environment	Controlled production access.
R9	Rollback	Versioned artifacts	Faster recovery.
R10	Monitoring	Health checks + alerts	Rapid detection of deployment issues.

9.1 Expected Benefits

- Faster feedback to developers when a change introduces a defect.
- Reduced manual effort in building, testing, and deploying the application.
- Consistent software artifacts across environments.
- Improved traceability from production release back to a source commit.
- Higher confidence through automated regression and end-to-end testing.
- Improved security through automated vulnerability checks.
- Safer releases through staging validation and approval gates.
- Faster recovery through versioned artifacts and rollback.

10. CONCLUSION

The proposed CI/CD pipeline provides a structured DevOps approach for FoodRescue Connect. It connects source-code management, automated build, testing, quality analysis, security scanning, packaging, deployment, monitoring, and rollback into a repeatable workflow. This ensures that software changes are verified before they are promoted to users.

The pipeline begins with a developer commit or pull request and performs fast quality and unit checks. After a successful build, integration and security tests are executed. A versioned artifact is then deployed to the development environment and promoted to staging for broader testing. Production deployment is protected by an approval gate and is followed by health checks and monitoring.

The design is particularly suitable for FoodRescue Connect because the application contains workflows involving multiple roles and time-sensitive data. Automated testing helps protect important functions such as food donation, request processing, volunteer assignment, status tracking, and administrative access.

Security and reliability are incorporated throughout the pipeline rather than being treated as final activities. Secret management, dependency scanning, container scanning, branch protection, least-privilege permissions, monitoring, and rollback collectively reduce operational risk.

In conclusion, implementing the proposed CI/CD process can improve development speed, software quality, deployment consistency, and recovery capability. The design can be extended in a real project with infrastructure-as-code, automated database migrations, advanced observability, container orchestration, blue-green or canary releases, and more extensive performance testing.

REFERENCES

Pressman, R. S. and Maxim, B. R., Software Engineering: A Practitioner's Approach, McGraw-Hill.

Sommerville, I., Software Engineering, Pearson.

GitHub Documentation – GitHub Actions and repository workflows.

Docker Documentation – Container images and application packaging.

ISTQB, Certified Tester Foundation Level – Software Testing principles and practices.

ISO/IEC/IEEE 29119, Software Testing standards and guidelines.

FoodRescue Connect project problem statement and assigned CI/CD Pipeline Design instructions.